Three-Week Interview Roadmap

LLM inference optimization, re-grounded in the `llm-inference-optimization` repository

Target: research master's admissions interviews, systems side (kernels, serving, memory, quantization). Bar: defence, not recognition. Every claim below carries a number, and every number marked `VERIFIED` was read out of a file in this repo, not recalled. Hardware of record: one NVIDIA RTX 3060 12GB (Ampere, sm_86). Model of record: Qwen/Qwen2.5-1.5B, fp16.

HOW TO READ THIS DOCUMENT

§1 is a guided tour of the repository, written for someone who did not build it. Read it first; nothing else in this document makes sense without it. **§2** is the measurement inventory: every experiment, its headline number, and the interview question it answers. **§3** audits the five results that were carried in from memory. **§4** is the revised three-week plan with a changelog against the previous version. **§5** is the four anchor answers, rewritten as quotable paragraphs. **§6** names what is deliberately not being learned.

1. What this repository actually is

This is a measurement artifact, not a library. Nothing in it is meant to be imported. Every script exists to produce one number, and every number exists to settle one argument about why LLM inference is slow. Read it in that order: the argument first, then the number, then the code that produced it.

1.1 The one-sentence thesis

At batch size 1, generating a token from a 1.5B-parameter model on a 12GB consumer GPU is *not* limited by arithmetic. It is limited by how many bytes must cross the memory bus, and by how much of the time the GPU spends doing nothing at all while Python decides what to launch next. Everything in this repo is a measurement of one of those two terms, or of a technique that shrinks one of them.

1.2 The directory map, annotated

Path	What it is, and why you would open it
`CLAUDE.md`	Persistent project context. Hardware constraints, the fp16-not-bf16 rule, the style rules. Read once.
`docs/phase1.md`	The original task breakdown (harness → HF baseline → vLLM → OOM sweep). Contains the *measurement rules* in §"Shared measurement definitions" — these are the most reusable paragraphs in the repo. Note: byte-identical to `weeks/week2.md`, and still says "Colab T4" throughout. Stale on hardware, correct on method.
`docs/vastai.md`	The compute record. Why there are two Python environments and why `pip install vllm` into the wrong one destroys the baseline.
`scripts/bench_common.py`	**Start here in the code.** 253 lines. The `CudaTimer`, the `VramSnapshot`, the CSV row schema, and `kv_cache_bytes()`. Everything else imports it.
`scripts/baseline_hf.py`	The control. A hand-written prefill + decode loop (deliberately *not* `.generate()`) so the two phases are timed apart. Also runs the NF4 4-bit variant through the identical loop.
`scripts/bench_vllm.py`	Same workload through vLLM's offline `LLM` class. Splits prefill from decode by timing `max_tokens=1` and `max_tokens=256` and subtracting.
`scripts/oom_sweep.py`	The headline experiment. 32-token prompt, then decode one token at a time for six hours until the card dies.
`scripts/bench_vllm_batch.py` `scripts/plot_batch_sweep.py`	Concurrency sweep and its plots. Run twice: a realistic KV pool and a deliberately starved one, to separate two different plateaus.

`scripts/profile_decode.py`
`scripts/profile_vllm.py`
`scripts/analyze_trace.py`
	The profiling pass. Captures `torch.profiler` traces of one decode step and reduces each to three numbers: kernels, CPU launches, device idle.

`scripts/measure_bandwidth.py`	Measures this specific card's theoretical *and* achievable bandwidth. Exists because "% of peak" is meaningless until you say which peak.
`docs/*-results.md`	Four writeups, one per experiment. These are the interview ammunition. `profiling.md` is the strongest single document in the repo.
`results/*.csv`	Raw rows. Never trust a doc over a CSV — §3 below found three places where they disagree.
`blog/draft.md`	The public-facing narrative. Currently the most up-to-date synthesis of every result.
`weeks/week3-6.md`	The working log. `week6.md` is the honest one: it records two weeks where all the input work happened and none of the output work did.

1.3 The five measurement rules the whole repo obeys

These are in `docs/phase1.md` and encoded in `bench_common.py`. An interviewer who asks "how do you know your benchmark isn't lying to you" is asking for exactly this list.

1. **Prefill and decode are timed separately and never blended.** They hit different walls; averaging them produces a number that describes neither.
2. **`torch.cuda.synchronize()` immediately before every timer read.** Kernel launches are asynchronous, so an unsynchronized timer measures *launch* time, not execution time. On a decode step this reports microseconds where the truth is tens of milliseconds. This is the single most common way an inference benchmark lies.
3. **Warmup run, discarded.** The first call pays CUDA context init and kernel autotuning.

4. **Track `memory_allocated` and `memory_reserved` separately, and reset peak stats before each measured run.** Allocated is what live tensors hold; reserved is what PyTorch's caching allocator has taken from the driver. The gap between them is where the OOM story lives.
5. **Fairness: change exactly one thing.** Identical model, prompt token ids, `max_new_tokens`, greedy decoding, batch size. The vLLM run is fed the same byte-identical prompt ids as the HF run.

> **THE ONE THAT GETS ASKED**
>
> Rule 2 is the one interviewers probe, because it is the one everyone gets wrong. The reason it matters: `model(input_ids)` returns a tensor handle almost immediately — PyTorch has allocated the output buffer and enqueued the kernels, but the GPU may not have started. The Python object is valid; the numbers in it are not there yet. Anything that crosses the GPU→CPU boundary (`.item()`, `.cpu()`, `print()`) forces an implicit sync. An accidental `.item()` inside a timing loop therefore serialises the pipeline and changes what you are measuring. See `docs/notes-cuda-memory-and-timing.md`.

1.4 The two environments, and why

vLLM is binary-tied to a specific torch build. The Vast.ai NGC image ships torch 2.12.0+cu130; vLLM 0.23 bundles torch 2.11.0+cu130. A naive `pip install vllm` into the baseline environment silently replaces the torch that every baseline number was measured against, which invalidates the control. Hence:

```
Environment A  (baseline, OOM sweep, profiling of HF)
  preinstalled torch 2.12.0+cu130, transformers 4.46.3, accelerate 0.34.2, bitsandbytes

Environment B  (vLLM, batching sweep)  /workspace/vllm-env
  vLLM 0.23.0 (v1 engine) + torch 2.11.0+cu130,  installed with
  uv pip install vllm --torch-backend=cu130
```

The `transformers` pin is not cosmetic. Version 4.44.2 registered per-layer `cos_cached` / `sin_cached` RoPE buffers sized to the full 131,072-token context: `28 layers × 2 tables × 131072 × 128 × 2 bytes = 1792 MiB` of VRAM parked before a single token is generated. 4.46+ hoists rotary to one model-level module that computes cos/sin on the fly. Resident weights dropped 4737 → 2945 MiB. That 1.79 GB was worth roughly 64k tokens of context in the OOM sweep — a library version silently eating a fifth of the headline experiment.

SOURCES FOR §1

- vLLM docs, offline inference and engine arguments
- Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention* — arXiv:2309.06180
- PyTorch, CUDA semantics: memory management (the allocated-vs-reserved model)

2. Measurement inventory: experiment → number → question

Every row was read out of `results/*.csv` or the corresponding `docs/*.md`. Where the CSV and the doc disagree, the CSV wins and the discrepancy is flagged in §3.

Experiment	Headline number	The interview question it answers
HF fp16 baseline `results/baseline_hf.csv`	Prefill 122 ms = **4205 tok/s**. Decode **28.1 tok/s** (settled; two early runs at 18.0 and 23.2). Weights 2945.29 MiB, peak alloc 3133.23 MiB.	"Why is prefill 150× faster per token than decode on the same model and card?" → one weight read amortised over 512 positions vs one weight read per single token.
vLLM baseline `results/baseline_vllm.csv`	Prefill 124 ms = **4114 tok/s** (1.02×). Decode **79.90 tok/s** (2.8×). Run-to-run spread **0.03%** vs HF's ~30%.	"Where does a serving engine actually win?" → not prefill (both call the same cuBLAS GEMMs), only decode. And: "why is vLLM's timing so much more reproducible?" → CUDA graph replay is deterministic.
Decode profiling 2×2 `results/profile_summary.csv`	Of 28.09 ms/token saved: **25.34 ms (90%) is device idle removed**, 2.60 ms (9%) is faster device work. HF step is **62.3% idle**. Launches/step 1198 → 17.	*The strongest answer in the repo.* "Why is CUDA-graph capture worth anything if it does not change the kernels?" → it does not change the kernels; it removes the CPU from the loop.
KV-cache OOM sweep `results/oom_sweep.csv`	Dies at **123,565 tokens** of total context. Measured slope **62 KiB/token** against an analytical **28 KiB/token** (2.21×). At OOM: 10,437 MiB allocated, 11,764 MiB reserved, on a 12,288 MiB card.	"Walk me through a memory problem you debugged." → two named, quantified mechanisms: `torch.cat` copy-on-grow, then external fragmentation.
Decode decay under context same CSV	**30.03 → 2.93 tok/s** from 1k to 123k context, a 10.3× slowdown.	"What happens to per-token latency as context grows, and why?" → three memory terms, two of which are linear in context.
NF4 quantization `results/baseline_hf.csv`	Weights 2945 → **1099 MiB** (2.68×, not 2×). Decode 28.1 → **12.4 tok/s**, i.e. **2.3× slower**. Prefill 122 → 140 ms (13% slower only).	"Weight-only quantization is a bandwidth play — so why did yours make decode slower?" → because bitsandbytes dequantizes to fp16 before the GEMM, so no bandwidth is saved where it counts.
vLLM batching sweep, 3060 `results/vllm_batch_sweep.csv`	Decode throughput **88.6 → 2696 tok/s** from batch 1 to 64 (30×), then flat to 2967 at batch 256 while TPOT climbs 11.3 → 86.3 ms. **Knee at 64.**	"What is the right batch size and how would you find it?" → the throughput/latency Pareto knee, with a fitted model behind it.
The knee is a model property 3090 cross-check, same CSV	3090 (2.6× bandwidth, 1.4× compute): every throughput number scales **2.4–3.4×**, but the **knee stays at 64** on both cards.	"How do you know your knee isn't a property of that one GPU?" → a stated, falsifiable prediction, tested on a second card, confirmed.
KV-admission wall constrained pool, same CSV	Pool starved to **25 sequences**: past batch 32 the `queued` flag flips, throughput pins at **~1000 tok/s** (a third of 2700), TPOT climbs 33 → 67 ms. Device VRAM **flat** throughout.	"How does a paged engine fail under memory pressure, versus a naive one?" → it queues instead of crashing, and the failure is invisible in `nvidia-smi`.
Bandwidth ceiling `scripts/measure_bandwidth.py`	Theoretical **360.0 GB/s** (NVML: 7501 MHz × 2 × 192 bit). Achievable **291.5 GB/s** = 81% of theoretical. vLLM decode hits **85% of achievable**; HF hits 26%.	"You said 68% of peak — which peak?" → the difference between a spec-sheet number and a measured ceiling, and why quoting the wrong one changes the conclusion.

Experiment	Headline number	The interview question it answers
Two-box variance `docs/profiling.md`	Two nominally identical 3060s, same VRAM: identical work ran **1.22× slower** on one, spread 1.4% across four runs. Cause still unexplained; the obvious memory-downclock hypothesis was measured and killed.	"Tell me about a measurement you could not explain." → a genuinely open result, honestly bounded, with the wrong hypothesis shown being falsified.

2.1 Model and hardware constants, verified

These are the numbers to have memorised. The parameter count below is *derived*, not quoted — see the verification in §2.2, because reconstructing it on a whiteboard is a standard interview exercise.

Quantity	Value	Where it comes from
Layers	28	every doc; used in `kv_cache_bytes()`
Query heads / KV heads	12 / 2	GQA, group size 6
head_dim	128	12 × 128 = 1536 = d_model
d_model	1536	confirmed by `docs/baseline-hf-results.md` ("K=1536")
intermediate (FFN)	8960	Qwen2.5-1.5B config; validated by the param sum below
vocab	151936	same; tied input/output embeddings
Parameters	1,543,714,304	**derived and confirmed**, §2.2
fp16 weight floor	2944.40 MiB	1,543,714,304 × 2 bytes
Measured resident weights	2945.29 MiB	CSV; 0.89 MiB over floor
KV bytes per token	28,672 B = 28 KiB	2 × 28 × 2 × 128 × 2
GPU VRAM	12,288 MiB	RTX 3060 12GB
Bandwidth, theoretical / achievable	360.0 / 291.5 GB/s	`measure_bandwidth.py`
Peak fp16 tensor (fp32 acc / fp16 acc)	~51 / ~100 TFLOPS	sm_86 spec
Roofline ridge point	142 FLOP/byte	51e12 / 360e9

VERIFICATION — THE PARAMETER COUNT SUMS TO THE PUBLISHED TOTAL

```
Embedding (tied, counted once)
  151,936 x 1,536                              =   233,373,696

Per layer:
  q_proj   1536 x 1536 + bias 1536             =     2,360,832
  k_proj   1536 x  256 + bias  256             =       393,472    (2 KV heads x 128 = 256)
  v_proj   1536 x  256 + bias  256             =       393,472
  o_proj   1536 x 1536, no bias                =     2,359,296
  gate_proj 1536 x 8960                        =    13,762,560
  up_proj   1536 x 8960                        =    13,762,560
  down_proj 8960 x 1536                        =    13,762,560
  2 x RMSNorm weight (1536 each)               =         3,072
                                       per layer =    46,797,824

  x 28 layers                                  = 1,310,339,072
  + embedding                                  =   233,373,696
  + final RMSNorm                              =         1,536
                                         TOTAL = 1,543,714,304   <-- matches exactly

fp16 bytes = 1,543,714,304 x 2 = 3,087,428,608 B
           = 3,087,428,608 / 1024^2 = 2944.40 MiB
measured resident                    = 2945.29 MiB   (0.89 MiB over: inv_freq buffers + allocator round
```

Two things to notice and be able to say out loud. First, the **embedding table is 15.1% of the whole model** (233M of 1.544B) and is pure lookup: zero arithmetic, all memory traffic. Second, the **MLP is 88% of each layer's parameters** (41.29M of 46.80M) while attention is 11.8%. Most of the weight traffic is MLP; almost all of the systems complexity is attention.

2.2 Where the repo is thin

Naming these is defensive preparation. An interviewer who reads the repo will find them.

Gap	Consequence, and the answer to give
No Phase 2 kernel gate. `docs/gate-phase2.md` is specified in `weeks/week5.md` and `week6.md` and does not exist.	The repo can explain *engine-level* performance in detail but has never diagnosed a single kernel in coalescing / shared-vs-global / occupancy terms. This is the largest single gap. Week 1 Days 3–4 below is rewritten to close it.
No FlashAttention measurement. vLLM's `FLASH_ATTN` backend is named in a log; nothing isolates it.	FlashAttention is the most likely deep-dive topic and the repo cannot back it with a number. It has to be carried by derivation alone. Week 2 Days 3–4 is unchanged and non-negotiable.
No quantization writeup. The NF4 result exists only as CSV rows plus a blog section; there is no `docs/quantization-results.md` matching the other three.	Answer stands on the blog section, which is good. Two hours in Week 3 makes it a proper doc.
No GPTQ / AWQ / SmoothQuant, no speculative decoding, no tensor parallelism run.	Single GPU, three weeks. Correct scope decision. Say so plainly.
No prefix caching / RadixAttention measurement — prefix caching is explicitly *disabled* in the vLLM benchmark so the second run cannot reuse the first run's prefill.	That is the right call for benchmark hygiene, and saying *why* it is disabled is itself a good answer.
`docs/phase1.md` **duplicates** `weeks/week2.md` **byte-for-byte** and both still say "Colab T4".	Cosmetic repo debt. Fix in 10 minutes; it is the first thing a careful reader notices.
Profiling window is 11–12 decode steps at sequence position ~512–525.	Already stated as a limitation in `docs/profiling.md`. The idle fraction should *fall* deep into a long generation, because attention kernels lengthen while launch count stays fixed. Untested. Volunteer this before being asked.

3. Audit of the five remembered results

Every one was treated as unverified and checked against files. Four confirm. One needs correcting. Several carry sub-claims that do *not* survive, and those matter more than the headlines because they are the ones that would be quoted wrongly.

CONFIRMED 1. Qwen2.5-1.5B, fp16, RTX 3060 12GB

Every CSV row, every doc. `gpu_name = NVIDIA GeForce RTX 3060`, `dtype = float16`. The one cross-card exception is the batching sweep, which also contains RTX 3090 rows — deliberately, as the falsifiable test of the knee claim.

CONFIRMED 2. KV-cache OOM cliff at ~123,565 tokens

Correct, with a bookkeeping subtlety worth knowing before someone catches you on it. The final CSV row reads `new_tokens = 123533`, `oom = True`, note `"CUDA OOM while decoding token 123533"`. The prompt was 32 tokens. **32 + 123,533 = 123,565**, so 123,565 is *total context* and 123,533 is *generated tokens*. Quote the total; know the split.

SUB-CLAIM CORRECTION: THE SLOPE IS 62 KIB/TOKEN, NOT 60

`docs/oom-results.md` says the measured line climbs at "~60 KB/token, roughly 2.15x the analytical 28 KB/token". Regressing the CSV directly:

```
peak_allocated at   1,000 tokens = 3,018.37 MiB
peak_allocated at 123,000 tokens = 10,405.06 MiB

slope = (10,405.06 - 3,018.37) MiB / 122,000 tokens
      = 0.060547 MiB/token = 63,487 bytes/token = 62.0 KiB/token

ratio to analytical = 63,487 / 28,672 = 2.21x    (doc says 2.15x)
```

Use **62 KiB/token, 2.21×**. Both the doc's rounding and the true value support the same argument, but if you are asked to reproduce the regression on paper, the CSV is what you will be reproducing.

The two mechanisms behind the gap, both verified numerically:

```
MECHANISM 1 -- torch.cat copy-on-grow doubles the resident cache
  HF's decode loop does past_kv = torch.cat([old, new_kv]) every single step.
  A tensor cannot grow in place: a fresh contiguous block for n+1 tokens is
  allocated and the old n copied in, so for the duration of the copy BOTH are live.
  Peak is therefore weights + 2 x KV, not weights + KV.

  check at 120,000 tokens:
    analytical KV = 28,672 x 120,000  = 3,281.25 MiB = 3.28 GiB
    weights + 2 x KV = 2,945 + 6,562  = 9,507 MiB
    measured peak allocated           = 10,226.89 MiB
    residual                          =   719 MiB  (attention scratch, grows with seq)

  slope check: d/dn (weights + 2KV) = 2 x 28 KiB = 56 KiB/token
               measured                           62 KiB/token   -- 6 KiB is the scratch term

MECHANISM 2 -- the crash is external fragmentation, not exhaustion
  at OOM:  allocated = 10,437 MiB
           reserved  = 11,764 MiB   <-- 1,327 MiB free INSIDE PyTorch's own pool
           card      = 12,288 MiB

  Those 1.3 GiB could not be handed out. The next cat needed ONE contiguous
  2 x KV-sized slab, and the free space was scattered in gaps between cache
  blocks already parked. Kernels index a flat buffer as base + i*stride; there
  is no page-table indirection making scattered VRAM look contiguous.
  Enough empty parking spaces in total for the bus, no unbroken stretch to fit it.

  Reserved plateaus at 11,732 MiB from ~43k tokens and steps to 11,764 near the end:
  that flat top is the caching allocator maxed out against the card, with the
  remaining ~520 MiB being the CUDA context.
```

CONFIRMED 3. Batching knee around batch 64

Realistic pool, median of 2 repeats: decode throughput 2696 tok/s at batch 64, then 2608 / 2879 / 2657 / 2967 at 96 / 128 / 192 / 256 — noise around a plateau, not growth. TPOT is flat to 64 (11.3 → 23.7 ms) then breaks (36.8 → 44.5 → 72.3 → 86.3 ms). The knee is where the fitted model puts it:

```
step_time(batch) = T_fixed + c1 x batch      T_fixed ~ 11 ms   (the shared 3.09 GB weight read)
throughput(batch) = batch / step_time                c1 ~ 0.20 ms/seq (that sequence's own KV read + GEMM sl

small batch:  T_fixed dominates  ->  throughput ~ batch / T_fixed, near-linear   (the free lunch)
large batch:  c1 x batch dominates -> throughput -> 1/c1 ~ 3000 tok/s            (the plateau, measured 2967
knee where they cross: 0.19 x batch ~ 11  ->  batch ~ 58        measured ~64
```

And the falsifiable prediction, tested: on an RTX 3090 (2.6× bandwidth, 1.4× compute) every throughput number scaled 2.4–3.4× but **the knee did not move**. TPOT flat through 64 on both cards, breaking at 96 on both. The knee is the weight-read-versus-KV-read crossover, which depends on the model's weight size and per-sequence KV — identical on both cards.

SUB-CLAIM TO RECONCILE BEFORE AN INTERVIEW: THREE DIFFERENT BATCH-1 VLLM NUMBERS

Source	batch-1 decode tok/s	Conditions
`baseline_vllm.csv`	79.90	median of 3, vLLM 0.23.0, box A
`profile_summary.csv`	79.6	12.56 ms/step, vLLM 0.25.1, box B
`vllm_batch_sweep.csv`	88.6	vLLM 0.23.0, same box, 11% higher

The 88.6 is flagged as unreconciled in `docs/batching-results.md` itself. **Quote ~80 tok/s** as the batch-1 vLLM figure and, if pressed, say the batching harness reads 11% higher and you have not yet isolated why (different point in the session, and the sweep's decode window excludes the first token differently). Volunteering an unreconciled 11% is a stronger signal than a number that happens to be convenient.

CONFIRMED 4. ~90% of vLLM's win is idle removed; baseline was 62% idle

Exact, from `results/profile_summary.csv`. This is the most defensible result in the repo, and the method is more impressive than the number.

run	compile	graphs	kernels	CPU launches	busy ms	step ms	idle
HF	n/a	n/a	1198	1198	15.31	40.65	62.3%
vllm-eager	no	no	356	356	13.02	25.22	48.4%
vllm-compile	yes	no	354	354	12.90	22.41	42.4%
vllm-graphonly	no	yes	385	17	12.77	12.57	~0%
vllm-graph	yes	yes	383	17	12.71	12.56	~0%

```
24.6 -> 79.6 tok/s = 3.24x        40.65 - 12.56 = 28.09 ms saved per token
  device idle removed   25.34 ms   90%
  faster device work     2.60 ms    9%

the 2x2 in step time (ms)              two predictions stated BEFORE the runs
              graphs OFF  graphs ON      graphs-only step == eager busy:   13.02 predicted, 12.57 measured (
compile OFF        25.22      12.57      graphs step      == compile busy: 12.90 predicted, 12.56 measured (
compile ON         22.41      12.56
compile saves       2.81       0.01    <-- NOT ADDITIVE. This is the interesting cell.
```

Three sub-results here are each worth an answer on their own:

- **The launch count, not the launch cost, is the problem.** HF pays 21.1 µs of idle per launch against an average kernel of 12.8 µs. That is not a large per-launch cost. It is paid 1198 times per token. Batch-1 decode kernels are matrix-vector products of a few microseconds each, so the dispatch chain above a kernel costs more than the kernel.
- **vLLM's per-launch cost is *higher* than HF's** — 34.3 µs vs 21.1 µs, because each vLLM step also runs scheduling, block-table management, slot mapping, and sampling. It still wins the total (12.20 ms idle vs 25.34 ms) because it pays more, far fewer times. Fewer expensive launches beat many cheap ones.
- **`torch.compile` reduces cost per launch, not launch count.** Kernel count moves 356 → 354 and busy time by 0.9%, yet it cuts 2.81 ms. The saving is visible in per-launch cost dropping 34.3 → 26.9 µs (22%). Inductor is removing Python and dispatcher work sitting *above* each launch.

THE METHODOLOGY POINT — REHEARSE THIS, IT IS WHAT SEPARATES YOU FROM SOMEONE WHO RAN A PROFILER

Every configuration is measured **twice**: a clean pass with no profiler yielding wall time `T`, and a profiled pass yielding device busy time `B`. The reported gap is `T - B`, each term taken from the run where it is trustworthy. Why this matters: CUPTI does not change what a kernel computes, so `B` survives profiling, but wall time does not — the HF profiled step ran **1.73× slower** than the clean one. Using the profiled pass's own wall time would have inflated the gap in exactly the direction the hypothesis predicts. That is how an analysis manufactures its own conclusion.

Two more deliberate choices: `with_stack`, `record_shapes` and `with_modules` are all **off**, because stack unwinding costs CPU time per dispatch and would tax HF (1198 dispatches) far harder than vLLM under graphs (17), inflating the very difference under test. And device events map to steps by **correlation id, not timestamp**, because a kernel runs after the launch that issued it, sometimes well after, so bucketing by timestamp misattributes work at every step boundary (<0.3% needed the timestamp fallback).

CORRECTED

5. NF4: "weights halved, decode got slower"

The direction is right, the magnitudes are both wrong. From `results/baseline_hf.csv`, 6 fp16 rows and 6 NF4 rows under identical config:

metric	fp16	NF4	change
weights VRAM	2945.29 MiB	1099.13 MiB	**2.68× smaller**, not 2×. 1846 MiB freed.
peak allocated	3133.23 MiB	1287.09 MiB	same 1846 MiB delta
decode	28.1 tok/s	12.4 tok/s	**2.26× slower**
prefill	122 ms (4205 tok/s)	140 ms (3655 tok/s)	only 13% slower

"Halved" understates it: NF4 stores 4 bits per weight plus block scales, so ~2.7×, and the freed 1846 MiB is the whole point. The asymmetry between prefill (−13%) and decode (−56%) is the actual result, and it is the same shape as the HF-vs-vLLM asymmetry, for a related reason:

WHY 4-BIT MADE DECODE SLOWER, IN THE FORM TO SAY OUT LOUD

The instinct is that decode is memory-bound and 4-bit weights are a quarter of the bytes, so decode should speed up. It does not, because **bitsandbytes does not run a native 4-bit matmul**. It reads the 4-bit weights, dequantizes them back to fp16, and runs the ordinary fp16 GEMM. The matmul therefore reads fp16-width data either way, so there is no bandwidth saving where it would count, and the dequant pass is strictly extra work.

In prefill that extra work is amortised over 512 positions and hidden behind real compute, so NF4 nearly keeps up (−13%). In decode, at batch 1 with arithmetic intensity ~1, there is nothing to amortise over and no compute to hide behind, so the dequant lands directly on per-token latency and more than doubles it.

The honest counterweight, which you must volunteer: **a native low-precision kernel that stayed in 4-bit through the matmul would cut real memory traffic and could flip decode the other way.** That is what AWQ/GPTQ kernels in vLLM do. The bitsandbytes dequant-to-fp16 path is a property of that implementation, not of weight-only quantization.

And the payoff, tied back to the headline experiment: 1846 MiB freed, at the OOM sweep's measured 62 KiB/token, is `1846 x 1024 / 62 ≈ 30,500` additional tokens of context before the same wall — bought by giving up decode throughput.

Two further corrections the memory list did not contain

HF DECODE IS ~28 TOK/S, NOT 18-23

`docs/baseline-hf-results.md` quotes "~18-23 tok/s", which was written when only two runs existed (18.00 and 23.18, 2026-06-25). Four later runs under identical config cluster tightly: **28.06, 28.32, 28.19, 28.05**. `blog/draft.md` already corrects this to "take ~28 tok/s as the settled figure". The doc did not get updated. **Quote ~28 tok/s**, and note the wide early spread is itself a finding about rented hardware (the same finding the 1.22× two-box result makes rigorously).

"~4× DECODE SPEEDUP" DEPENDS ON WHICH HF NUMBER YOU DIVIDE BY

```
79.9 / 18.0 = 4.4x     (earliest, slowest HF run)
79.9 / 23.2 = 3.4x
79.9 / 28.1 = 2.8x     <-- against the settled HF figure, same box
79.6 / 24.6 = 3.24x    <-- the profiling comparison, both runs on box B, versions pinned
```

Quote 3.2× and cite the profiling row, because it is the only comparison where both sides were measured on the same box in the same session. The "~4×" in `docs/baseline-vllm-results.md` divides a fast vLLM number by a slow HF number measured a day earlier, which is exactly the cross-run comparison the repo's own two-box result warns against. Being the person who catches that in their own repo is worth more than the extra 0.8×.

4. The revised three-week plan

4.1 Changelog against the previous roadmap

Change	Why the repo forced it
Week 1 Days 3–4 rewritten from "reading fluency" to "take the kernel gate in writing."	The biggest gap found. The repo explains engine-level performance beautifully and has *never* diagnosed a kernel. `weeks/week5.md` and `week6.md` both specify `docs/gate-phase2.md`; it does not exist. Everything else in the roadmap is revision of things you can already defend. This is the one item that is net-new capability, and Layer 4/5 is a ★ tier.
Week 1 Days 5–6 (roofline) cut from two days to one.	Already over-covered. Ridge point 142 FLOP/byte, MBU percentages, achievable-vs-theoretical ceilings, and the batching roofline (predicted B* 142–280 vs measured knee 64) are all in the repo with numbers. This is rehearsal, not learning. The freed day goes to the gate.
"Build nanoGPT by hand" downgraded from mandatory to optional.	Its stated purpose is making attention shapes muscle memory. You get the same thing faster by tracing a decode step through *your own* config (§5 of the Days 1–2 notes does exactly this), and yours comes with measured latency attached. Do it only if the Day 1–2 self-test exposes shape confusion.
Week 2 Days 1–2 (KV cache / PagedAttention) cut to one day.	The OOM experiment *is* this topic, measured, with two named mechanisms and a corrected prediction. You are not learning it, you are rehearsing it. The half-day saved goes to FlashAttention, which the repo cannot back with a number at all.
Week 2 Days 3–4 (FlashAttention) expanded to 2.5 days and made the week's centre.	Inverse of the above: highest interview probability, zero repo support. The only thing carrying it is your derivation. Add the specific bridge the repo *does* support: the softmax max-subtraction trick you already need for numerical stability is the seed of the online-softmax recurrence.
Week 3 Day 4 changed from tensor parallelism to a numbers-reconciliation and repo-repair day.	§3 found four live inconsistencies that would each get caught in an interview: the 62-vs-60 KiB slope, the 88.6-vs-79.9 batch-1 vLLM figure, the stale 18–23 tok/s HF number, the 4×-vs-3.2× framing. Tensor parallelism drops to a 45-minute read (you have one GPU; you cannot defend a TP claim with anything but a paper anyway).
Added a standing 30-minute daily block: "explain yesterday's topic in 90 seconds, unaided, out loud."	Named in the previous roadmap as "the habit that matters most" and then not scheduled. Anything unscheduled loses to reading, which feels more productive and is not.

4.2 Week 1 — the layers the repo does *not* already prove

Goal: close the one real capability gap (kernels) and make the architecture layer reflexive.
Exit bar: whiteboard attention shapes through one decode step of *your* config unaided; state the roofline ridge point and where decode sits; and hand someone two versions of a matmul kernel and explain which is faster in coalescing, shared-vs-global, and occupancy terms, with numbers.

Days	Content	Deliverable and the question it answers
1–2	Attention from scratch: the √d variance derivation, softmax max-subtraction, causal masking, multi-head shapes. RoPE mechanism, verified numerically. MHA → MQA → GQA as a KV-bytes story.	Covered in full by the companion document, *Week 1 Days 1–2 lecture notes*. Terminates in your own config: 28 KiB/token under GQA, 168 KiB/token under MHA, a 6× ratio. Answers: "walk me through a decode step with shapes"; "why does GQA shrink the KV cache but not the weights".
3–4 CHANGED	**The kernel gate, in writing.** GPU architecture (SMs, warps, memory hierarchy, coalescing, occupancy) is already watched; this converts it to a defensible artifact. Take naive vs tiled matmul from GPU MODE lecture 5 and write out, with numbers: (a) bytes read from global memory per output element under each, and the arithmetic intensity that follows; (b) which loads coalesce and which do not, at the level of what a single warp touches in one instruction; (c) the occupancy math for your tile size on sm_86 — shared memory per block, registers per thread, resulting blocks per SM. Then repeat on one kernel from *your own* HF decode trace.	Commit `docs/gate-phase2.md`. If any of the three needs hand-waving, that is the part to fix, not to write around. Answers: "pick a kernel and tell me why it is slow" — the single most common systems-interview probe and currently your weakest area.
5 SHORTENED	Roofline and arithmetic intensity, as rehearsal only. Read Pope et al., *Efficiently Scaling Transformer Inference* twice — it is worth more than everything else in this area combined for an academic interview. Then re-derive your own numbers cold.	Reproduce from memory: ridge 142 FLOP/byte; decode intensity ~1–2; prefill 4205 tok/s vs decode 28 tok/s = 150×; achievable ceiling 291.5 GB/s and why it is not 360.
6	Inference arithmetic: parameter counting, the 2×params-per-token FLOP rule and its caveat, MBU/MFU, the latency-throughput Pareto.	Reproduce the 1,543,714,304 sum on paper (§2.1). Answers: "estimate the memory bandwidth needed to serve this model at 100 tok/s."
7	Buffer. No Sundays.	—

WEEK 1 LINKS

- Vaswani et al., *Attention Is All You Need* — arXiv:1706.03762 → the √d scaling footnote is the thing to be able to derive
- Su et al., *RoFormer* (RoPE) — arXiv:2104.09864
- Ainslie et al., *GQA* — arXiv:2305.13245; Shazeer, *Fast Transformer Decoding* (MQA) — arXiv:1911.02150
- Pope et al., *Efficiently Scaling Transformer Inference* — arXiv:2211.05102 → read twice; the single highest-ROI paper for an academic systems interview
- Boehm, How to Optimize a CUDA Matmul Kernel → the gate on Days 3–4 is essentially this article, written back in your own words
- He, Making Deep Learning Go Brrrr From First Principles → the compute/memory/overhead trichotomy your profiling result is a measurement of
- kipply, Transformer Inference Arithmetic

4.3 Week 2 — KV cache, FlashAttention, serving

Goal: make the two topics an interviewer is most likely to go deep on unassailable.
Exit bar: derive the online-softmax rescale update on a whiteboard without notes, state why FlashAttention does nothing for batch-1 decode and what does instead, and tell the OOM story in under 90 seconds with four numbers in it.

Days	Content	Deliverable and the question it answers
1 SHORTENED	KV cache size formula; PagedAttention block tables; the three wastes the paper names (reservation, internal fragmentation, external fragmentation) mapped onto what you measured; RadixAttention / prefix caching in one paragraph.	**Rehearse the OOM story out loud until it is 90 seconds.** The mapping to make explicit: *external* fragmentation is literally the crash you watched — OOM with 1.3 GiB free inside the pool because no single gap was large enough. Answers: "walk me through a memory problem you've debugged."
2–4 EXPANDED	**FlashAttention, properly derived.** Online softmax: the running max and the rescale factor, derived so you can write the recurrence cold. Why it is IO-awareness and not a FLOPs reduction — the same FLOPs, but the P×P score matrix never touches HBM. Then the decode case: **why FlashAttention does nothing at batch 1** (the score matrix is 1×n, there is no tile to keep in SRAM and no quadratic term to avoid) and what does instead: **flash-decoding / split-KV**, which splits the KV sequence across SMs to recover parallelism when the query dimension is 1.	The single most likely deep-dive question, and the repo cannot support it with a number, so the derivation *is* the answer. One bridge you can make concrete: the max-subtraction trick you need anyway for softmax stability is exactly the seed of the online recurrence. One measured hook: your vLLM traces show `flash split-KV` kernels firing 28 times per decode step at 14.9–16.2 µs each — that is flash-decoding, in your own trace.
5–6	Continuous batching (Orca) and chunked prefill (Sarathi): the mechanism and the specific problem each solves. Metric vocabulary: TTFT, TPOT/ITL, p50/p95/p99, goodput. Then map your batching sweep onto that vocabulary.	**Practise explaining your knee in Orca/Sarathi language.** You have the rarest thing here: two distinct plateaus measured separately (a compute-efficiency ceiling at 2700 tok/s where everything runs, and a KV-admission ceiling at 1000 tok/s where the surplus queues) plus a cross-GPU confirmation that the knee is a model property. Answers: "how would you pick a batch size for a production SLO?"
7	Buffer.	—

WEEK 2 LINKS

- Dao et al., *FlashAttention* — arXiv:2205.14135; *FlashAttention-2* — arXiv:2307.08691
- Milakov & Gimelshein, *Online normalizer calculation for softmax* — arXiv:1805.02867 → short, and it is the actual recurrence
- Dao et al., Flash-Decoding for long-context inference → the batch-1 answer
- Kwon et al., *PagedAttention / vLLM* — arXiv:2309.06180
- Yu et al., *Orca: distributed serving with iteration-level scheduling* — OSDI '22
- Agrawal et al., *Sarathi-Serve* (chunked prefill) — arXiv:2403.02310
- Zheng et al., *SGLang / RadixAttention* — arXiv:2312.07104 → also your stated OSS target, so worth one careful read

4.4 Week 3 — quantization, speculative decoding, synthesis

Goal: cover the two ◆ differentiators at one-paragraph-of-mechanism depth, repair the repo's live inconsistencies, and rehearse.
Exit bar: for every ★ layer, state the concept, the equation, and the one experiment or paper result that proves it — unaided, under 90 seconds each. All four anchor answers delivered cold.

Days	Content	Deliverable and the question it answers
1–2	Quantization, one paragraph of mechanism each: GPTQ (Hessian-based error-compensating rounding), AWQ (activation-aware per-channel scaling, protect the salient 1%), SmoothQuant (migrate activation outliers into the weights so both sides are quantizable), LLM.int8() (the outlier-feature story and mixed-precision decomposition), NF4/QLoRA. Then the framing: weight-only quantization is fundamentally a *bandwidth* play, not a compute play.	**Anchor on your NF4 result** (§3.5): weights 2.68× smaller, decode 2.26× *slower*, prefill only 13% slower, 1846 MiB freed ≈ 30,500 more tokens of context. Answers the trap question — "you said it is a bandwidth play, so why did your 4-bit run get slower?" — with a measured answer and the correct caveat about native low-precision kernels.
3	Speculative decoding: draft-then-verify, why the rejection-sampling correction keeps the output distribution exactly unchanged (plain language, skip the proof), acceptance rate as the thing that decides whether it pays. One sentence each on Medusa (extra heads) and EAGLE (feature-level autoregression).	Connect it to your own roofline: speculation works because verifying k drafted tokens in one forward pass costs about the same as generating one, since decode is memory-bound and you are re-using the same weight read. That framing is *your* measurement (T_fixed ~ 11 ms, c1 ~ 0.20 ms/seq) applied to a technique you did not run.
4 CHANGED	**Reconciliation and repo repair.** Fix the four live inconsistencies from §3: update the HF decode figure to ~28 tok/s in `docs/baseline-hf-results.md`; correct the slope to 62 KiB/token in `docs/oom-results.md`; either reconcile or explicitly caveat the 88.6-vs-79.9 batch-1 vLLM gap; settle on 3.2× as the headline speedup with the reason. Write `docs/quantization-results.md` to match the other three. De-duplicate `docs/phase1.md` / `weeks/week2.md` and strip the stale T4 references. *Then* 45 minutes on tensor parallelism: split heads and FFN, two all-reduces per layer, why it wants NVLink; pipeline parallelism gets one sentence ("throughput not latency, has bubbles").	An interviewer who opens the repo will find these. Fixing them yourself, and being able to say "I found four places where a doc had drifted from its own CSV and here is what I changed", is a better research signal than any additional topic you could cram into this day.
5–6	Synthesis. For every ★ layer: concept, equation, and the one experiment or paper result that proves it. Then rehearse the four anchor answers (§5) cold, timed, out loud.	The synthesis table is the artifact. If a layer has no proving experiment or result, that is the layer to spend the buffer day on.
7	Buffer.	—

WEEK 3 LINKS

- Frantar et al., *GPTQ* — arXiv:2210.17323; Lin et al., *AWQ* — arXiv:2306.00978

- Xiao et al., *SmoothQuant* — arXiv:2211.10438; Dettmers et al., *LLM.int8()* — arXiv:2208.07339; *QLoRA* (NF4) — arXiv:2305.14314
- Leviathan et al., *Fast Inference via Speculative Decoding* — arXiv:2211.17192; *Medusa* — arXiv:2401.10774; *EAGLE* — arXiv:2401.15077
- Shoeybi et al., *Megatron-LM* (tensor parallel) — arXiv:1909.08053
- Weng, Large Transformer Model Inference Optimization → best single synthesis pass for Days 5–6
- gpt-fast → ~1000 lines that do everything your profiling result says matters; skim as confirmation

5. The four anchor answers, rewritten with verified numbers

Almost any question routes back to one of these. Each is written to be said out loud in 60–90 seconds. The bolded numbers are the ones to land on.

> **ANCHOR 1 — THE FLASHATTENTION DERIVATION (NO REPO NUMBER; CARRIED BY DERIVATION)**
>
> Standard attention computes the full `N×N` score matrix, writes it to HBM, reads it back for softmax, writes it again, reads it again for the `V` multiply. The FLOPs are unavoidable; the HBM round-trips are not. FlashAttention tiles Q, K and V into blocks that fit in on-chip SRAM and never materialises the score matrix in HBM at all — same arithmetic, an order of magnitude less memory traffic. It is an IO-complexity result, not a FLOPs result.
>
> The obstacle is that softmax needs its whole row for the denominator, which looks incompatible with tiling. Online softmax fixes it: carry a running max `m` and a running sum `ℓ`; when a new tile arrives with a larger max, rescale the accumulated output and the running sum by `exp(m_old - m_new)` before adding the new block's contribution. The result is exact, not approximate. That rescale factor is the entire trick, and it is the same max-subtraction you already need for plain numerical stability — overflow protection turned into an incremental algorithm.
>
> **The part that shows you have thought about inference specifically:** FlashAttention does essentially nothing for batch-1 decode. The score matrix is `1×n`, so there is no quadratic term to avoid and no tile worth keeping in SRAM. Decode's problem is the opposite — not enough parallel work to fill the GPU. The answer there is **flash-decoding / split-KV**, which splits the KV sequence across SMs and combines the partial softmaxes with the same rescale rule. In my own vLLM decode traces that kernel is visible: **28 flash split-KV launches per step at ~15 µs each**.

ANCHOR 2 — THE KV-CACHE OOM STORY

I wanted to watch the KV cache kill a GPU rather than compute that it would, so I ran Qwen2.5-1.5B in fp16 on a 12GB RTX 3060 with a 32-token prompt and grew the context purely by decoding, one token at a time, logging VRAM every 1000 tokens. Six hours of sequential decode. It died at **123,565 tokens** of context.

The analytical KV size is `2 × 28 layers × 2 KV heads × 128 head_dim × 2 bytes = 28,672 bytes = **28 KiB per token**`. Note it is the 2 KV heads that size the cache, not the 12 query heads — that is what GQA buys. But the measured curve climbed at **62 KiB/token, 2.21× the prediction**, and sat above it the whole way.

That gap is a mechanism, not overhead. **HuggingFace reallocates the entire KV cache on every decode step**: `torch.cat([old, new])` cannot grow a tensor in place, so it allocates a fresh contiguous block for `n+1` tokens and copies the old `n` in, which means both copies are live during the copy. Peak is `weights + 2×KV`. At 120k tokens that predicts **9,507 MiB** and I measured **10,227**, with the residual ~700 MiB being attention scratch that itself grows with sequence length.

And the crash is **fragmentation, not exhaustion**. At OOM I had 10,437 MiB allocated but 11,764 MiB reserved — **1.3 GiB free inside PyTorch's own pool that could not be handed out**, because the next `cat` needed one contiguous 2×KV slab and the free space was scattered between cache blocks already parked. Enough empty parking spaces for the bus, no unbroken stretch to fit it.

I also got the prediction wrong and kept the wrong guess in the writeup. Beforehand I estimated the card would die near 30k tokens. It reached 123k. The mechanism I named was right and my arithmetic was pessimistic. Both mechanisms are exactly what PagedAttention removes: fixed-size blocks with a block table, so there is no `cat`, no second copy, and any freed block fits any future need. That is the same paper's "external fragmentation" waste category, and I had watched it happen before I read the name for it.

ANCHOR 3 — THE BATCHING KNEE

Batch 1 wastes the GPU: the decode step reads all **3.09 GB** of weights to advance one sequence by one token, arithmetic intensity ~1, tensor cores idle. So I swept concurrency in vLLM from 1 to 256 on the same 512-token prompt. Decode throughput went **88.6 → 2696 tok/s from batch 1 to 64** — a 30× gain — then essentially stopped, reaching only 2967 at batch 256 while per-token latency climbed **11.3 → 86.3 ms**. The knee is 64.

The shape falls out of one equation. A decode step costs `T_fixed + c1 × batch`, where `T_fixed ≈ 11 ms` is the shared weight read paid once regardless of batch, and `c1 ≈ 0.20 ms` is what each added sequence costs — its own KV read plus its slice of the GEMM. Throughput is `batch / (T_fixed + c1·batch)`: near-linear while the fixed cost dominates, asymptotic to `1/c1 ≈ 3000 tok/s` once it does not. The knee is where they cross, at batch ~58 predicted against ~64 measured.

The plateau is not what you would guess. At batch 256 the card is at ~9 TFLOPS (16% of its ~51 TFLOPS peak) and ~90 GB/s (25% of 360). *Neither roofline is saturated*, yet throughput will not rise. The reason is operation shape: batched decode is a skinny GEMM with M = batch, which is occupancy-bound and tops out far below tensor-core peak, and the paged KV reads are scattered. The plateau is the ceiling of those specific kernels, not of the GPU. The naive roofline predicts an ideal batch of **142** (or 280 at fp16-accumulate peak) against a measured **64**, and that 2–4× gap is itself the result.

I made this falsifiable. I predicted the knee is a property of the model, not the card, because it is the weight-read-versus-KV-read crossover. Rerunning on an RTX 3090 — 2.6× the bandwidth, 1.4× the compute — every throughput number scaled 2.4–3.4× and **the knee stayed at 64**, with TPOT flat through 64 and breaking at 96 on both cards. I got the plateau height wrong: I predicted 6–7k tok/s from bandwidth alone and measured 10,100, because the 3090's extra compute also lifts the skinny-GEMM ceiling. So the knee location is a model property and the plateau height depends on both axes.

ANCHOR 4 — THE HF-VS-VLLM PROFILING RESULT

vLLM decodes ~3.2× faster than HuggingFace at batch 1 on the same card and model. The usual explanation is "CUDA graphs and fused kernels", which is an argument, not a measurement, so I profiled it as a **2×2 ablation** over `torch.compile` and CUDA-graph capture, plus the HF baseline.

Headline: of the **28.09 ms per token** that separates them, **25.34 ms (90%) is device idle removed and 2.60 ms (9%) is faster device work.** The gap is a scheduling problem, and now there is a number behind that sentence. The HF decode step fires **1198 kernels with 1198 CPU launches and is 62.3% device-idle**. Per-launch idle is only 21.1 µs against an average kernel of 12.8 µs — it is not that a launch is expensive, it is that it is paid 1198 times, and batch-1 decode kernels are matrix-vector products of a few microseconds each, so the dispatch chain above a kernel costs more than the kernel.

The ablation separated two things the usual story conflates. **Fusion is vLLM's hand-written kernels, before any compiler runs:** with compile and graphs off on both sides, kernel count falls 1198 → 356 (3.4×) — fused RMSNorm, fused rotary, fused SiLU-and-multiply, paged FlashAttention. **CUDA graphs do not change the kernels at all:** counts are identical with graphs on and off, but CPU launches fall **356 → 17** and idle goes to zero. I stated the prediction before running it — if graphs remove idle and nothing else, the graphs-on step should equal the graphs-off *busy* time. Predicted 13.02 and 12.90 ms; measured 12.57 and 12.56. Within 3.5% and 2.6%.

The result I did not predict is the most useful one: the two optimizations are not additive. `torch.compile` saves 2.81 ms with graphs off and **0.01 ms with graphs on**. Once graphs have taken the CPU off the critical path, making the CPU's per-launch work cheaper is worth nothing. Measuring only the shipping default against plain eager would have credited graphs with a fusion win, or the reverse, with no way to tell.

And it closes the question of what is left. Measured against this card's *achievable* read bandwidth — 291.5 GB/s, which I measured rather than taking the 360 GB/s spec figure — vLLM's decode step runs at **85%**. There is almost nothing left to win from scheduling at batch 1. The next lever has to be arithmetic intensity, which is the batching experiment.

6. What is deliberately not being learned, and why

Naming these confidently is a positive signal. "Aware of it, haven't gone deep" is a correct and well-calibrated answer; hedging or bluffing is not. All ○-tier items are cut in full.

Cut	Reason, in the form to say it
MoE routing and expert parallelism	Single-GPU, dense-model artifact. Everything I measured is about weight-read amortisation, which MoE changes qualitatively. I would rather be precise about dense inference than vague about both.
Ring Attention, sequence parallelism	Multi-GPU long-context technique. I have one card and my long-context result is a memory result, not a distribution result.
FP8 and FlashAttention-3	Hopper-class features. My card is Ampere sm_86; I could recite them but not measure them, and reciting is the failure mode I am avoiding.
MLA (DeepSeek) beyond one sentence	One sentence is enough: it compresses K and V into a shared low-rank latent, so it attacks the same KV-bytes term as GQA but harder. I can place it on the MHA→MQA→GQA axis; I have not derived it.
P/D disaggregation (DistServe, Mooncake)	Multi-node serving architecture. I understand the motivation directly from my own data — prefill is compute-bound at 4205 tok/s and decode is memory-bound at 28, so co-locating them on one device means one phase always interferes with the other — but I have not studied the systems.
Structured/constrained decoding, multi-LoRA	Serving features orthogonal to the performance question this artifact is about.
Training-side anything — ZeRO, FSDP, activation checkpointing, optimizer sharding	Out of scope by construction. Inference has no backward pass, no optimizer state, and no gradient memory, which is why the KV cache is the dominant growing tenant rather than activations.
Writing a production CUDA kernel	Week 1 Days 3–4 is *reading and diagnosing* a kernel, not writing one. Three weeks buys diagnosis; it does not buy authorship, and claiming authorship I cannot demonstrate is worse than claiming neither.

7. The habit that decides whether any of this works

End every topic by explaining it out loud, unaided, in under 90 seconds. If you cannot, **re-derive it from scratch rather than rereading**. Rereading produces recognition; re-derivation produces defence, and only one of those survives being interrupted with "wait, why is it √d and not d?"

Schedule it. The previous roadmap named this as the most important habit and then left it unscheduled, which is how it loses to reading. Thirty minutes at the start of each day, on yesterday's topic, timed. The measure of a day is not what you read; it is what you can now say cold.

Companion document: *Week 1, Days 1–2 — Attention, RoPE, and MHA→MQA→GQA*, which covers the Week 1 Days 1–2 unit in full and builds every architectural claim out of this repository's measured configuration.